



DIPLOMADO

Desarrollo de sistemas con tecnología Java

Módulo7

Persistencia con Spring Data

M. en C. Jesús Hernández Cabrera

jesushernandezls7@aragon.unam.mx



JPA

- JPA es una especificación de Java, es decir, es un estándar para un framework ORM.
- JPA menciona las reglas que se deben seguir para que pueda existir la interacción con las tablas de la base de datos en forma de objetos
- En otras palabras, JPA es la teoría y algunas de las implementaciones de esta teoría son: Hibernate, EclipseLink, TopLink.

JPA

- JPA usa anotaciones para evitar usar de manera nativa sentencias SQL.
 - @Entity
 - indica a una clase de java que está representando una tabla de BD.
 - @Table
 - recibe el nombre de la tabla a la cual está mapeando la clase.
 - @column
 - se asigna a los atributos de la clase, no es obligatoria, se indica sólo cuando el nombre de la columna es diferente al nombre del atributo de la tabla.

JPA

- JPA usa anotaciones para evitar usar de manera nativa sentencias SQL.
 - @id y @EmbeddedID
 - es el atributo como llave primaria de la tabla dentro de la clase. @id se utiliza cuando es llave primaria sencilla y @EmbeddedID cuando es una llave primaria compuesta
 - @GeneratedValue
 - permite generar automáticamente valores para atributos de la db.
 - @OneToMany y @ManyToOne
 - permite relacionar diferentes entidades/clases.
 - También se hace uso de las anotaciones @JoinColumn.

Spring Data Repositories

- Permiten ahorrar código y tiempo de implementación al poder realizar operaciones en la base de datos sin sentencias SQL.
- Existen diferentes repositorios en Spring Data
 - CrudRepository: para realizar operaciones CRUD.
 - PageableAndSortingRepository: además de las operaciones CRUD, se puede paginar y ordenar.
 - JpaRepository: agregó tareas específicas a las anteriores, como flush.

Spring Data Repositories

- A la clase que interactúa con la base de datos mediante una instancia de un repositorio, se le debe agregar alguna de las siguientes anotaciones:
 - `@Repository`
 - le indica a la clase que es la encargada de interactuar con la bd.
 - `@Component`
 - le indica que es un componente de spring.

Query Methods

- Cuando se necesitan consultas y el repositorio de Spring Data no puede hacer estas consultas, entonces los Query Method proveen la posibilidad de generar consultas mediante el nombre de los métodos.
- Son compatibles con programación funcional de Java.
- En lugar de usar *Query Methods*, se puede usar la anotación
 - *@Query*
 - para escribir consultas (select * from tabla where condición).

Entidades

@Entity

Alumno

@matricula

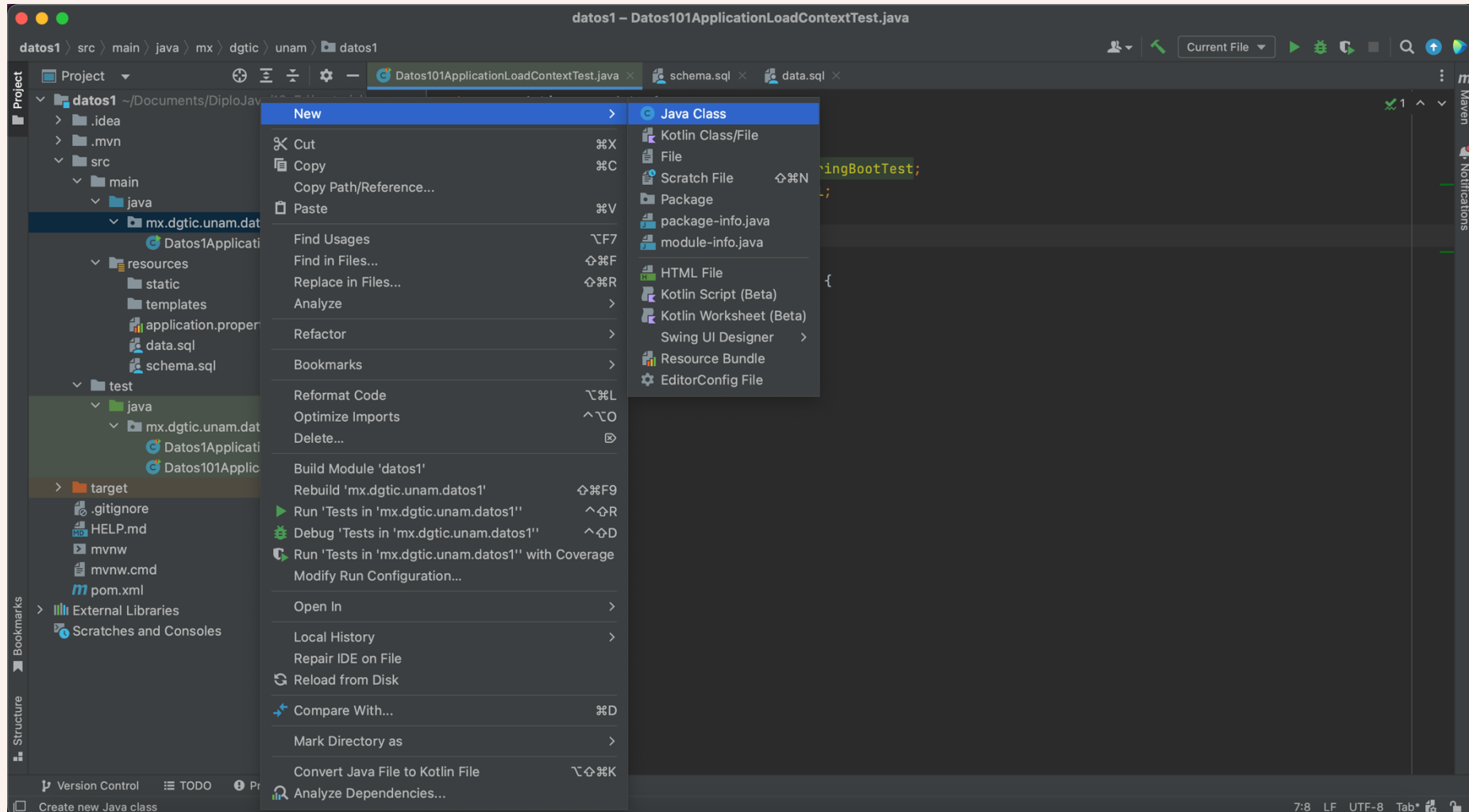
nombre

paterno

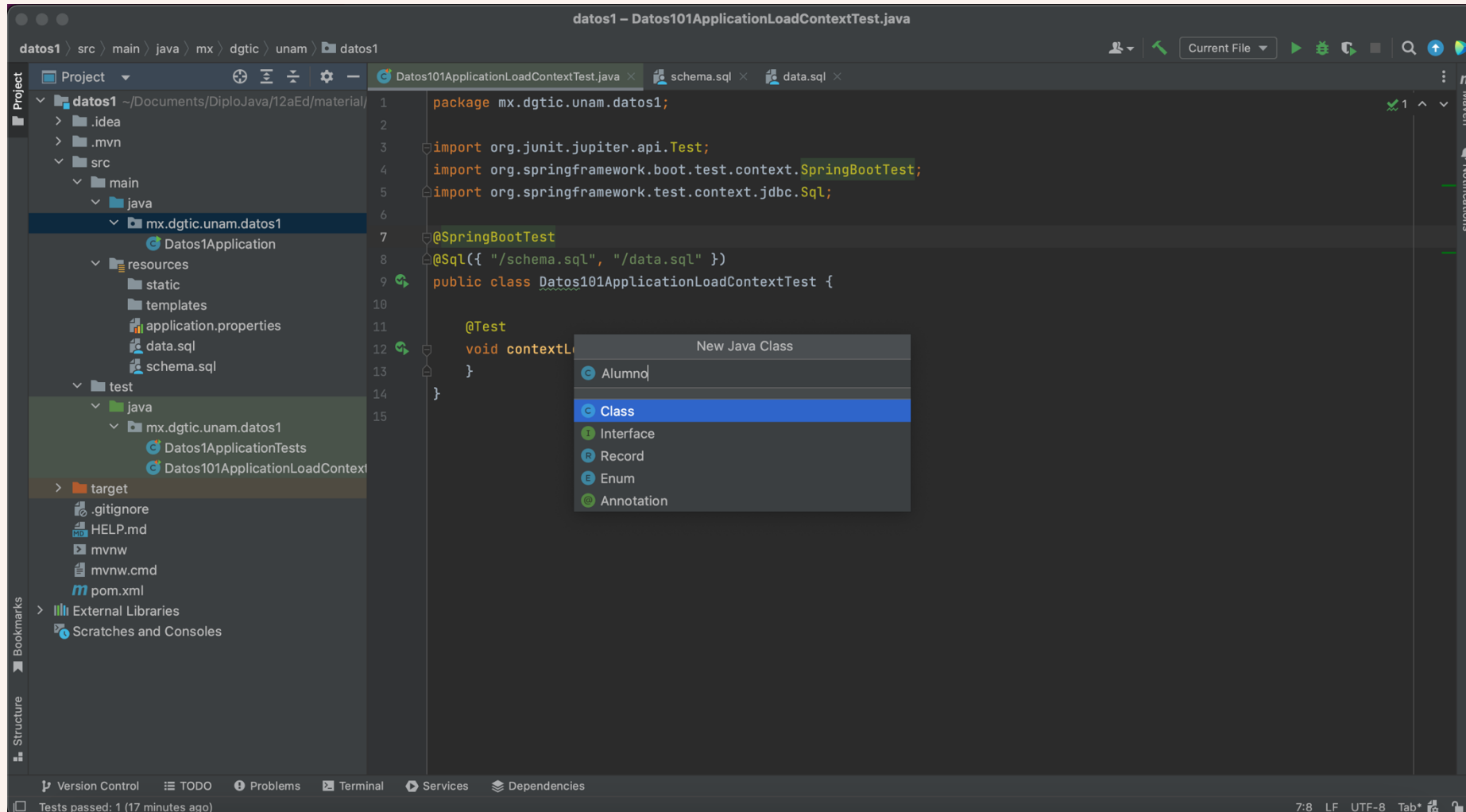
fnac

estatura

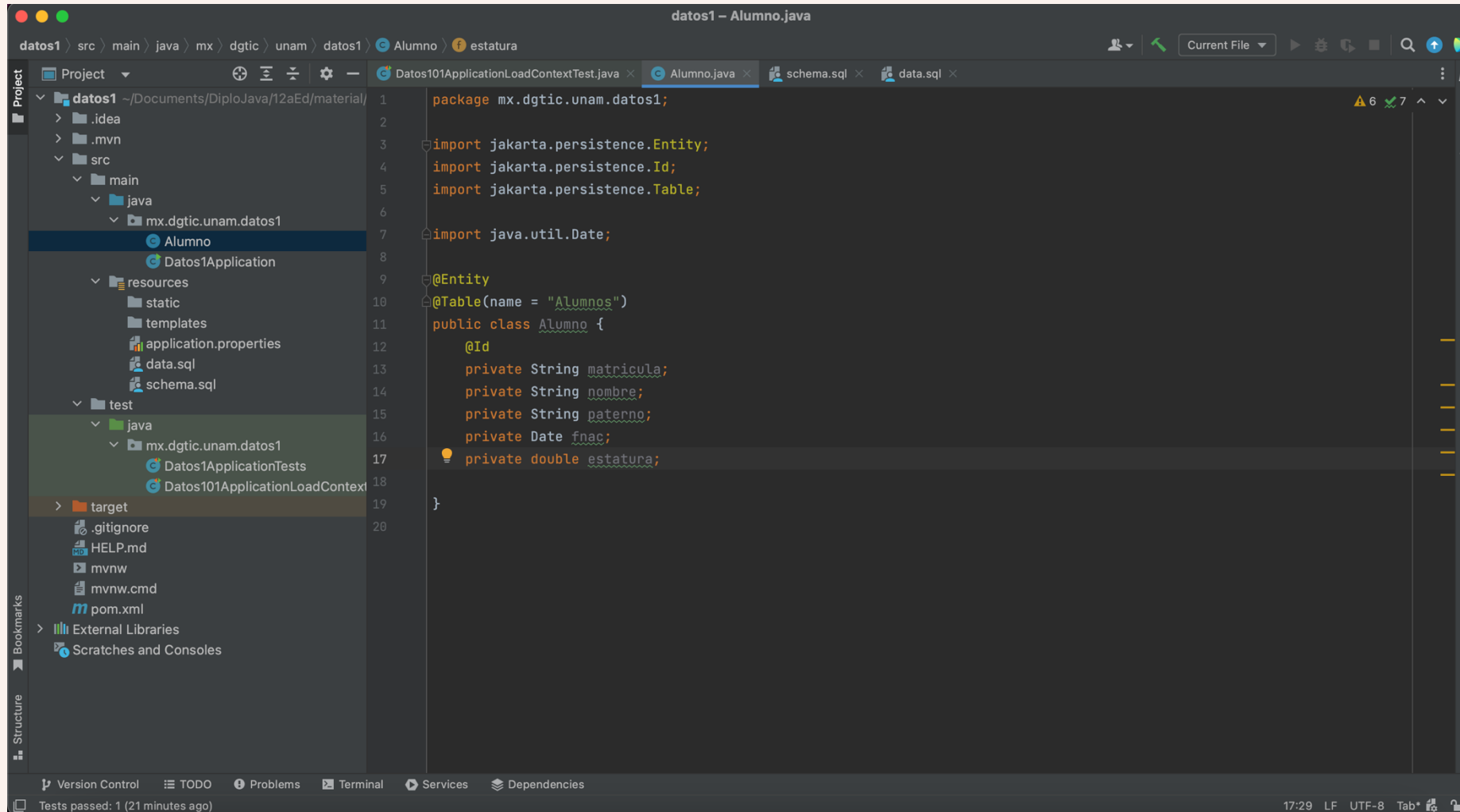
Generar una entidad



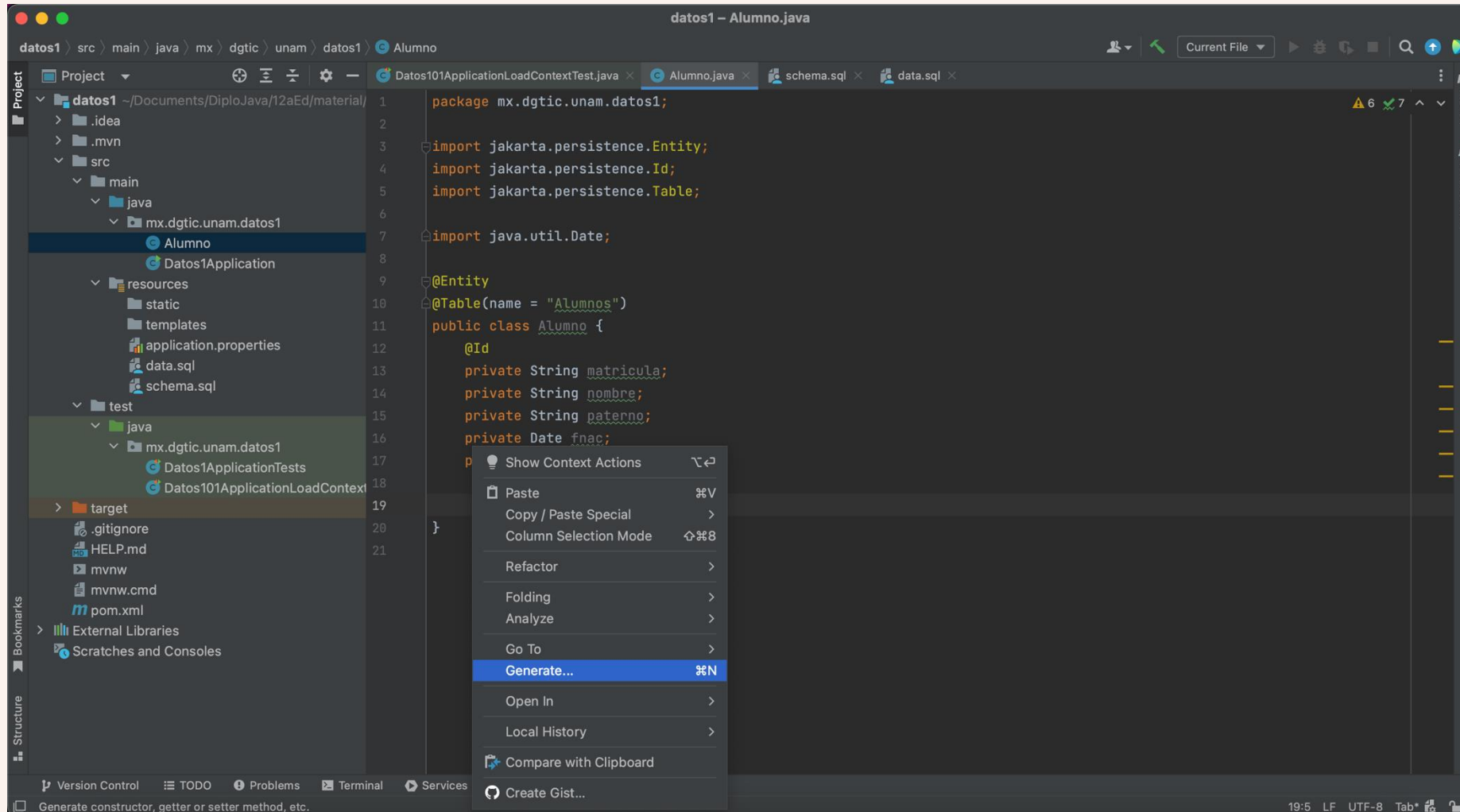
Generar una entidad



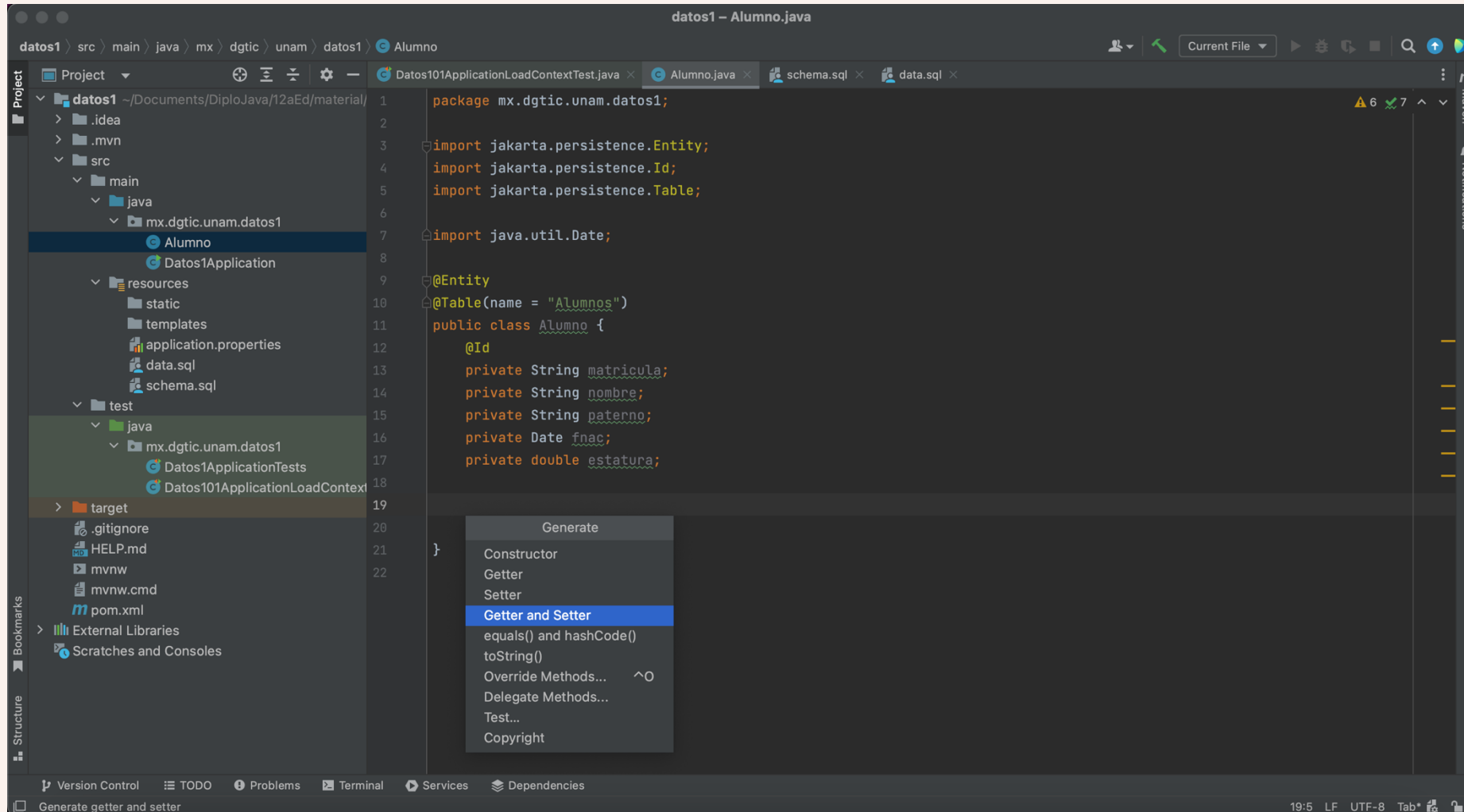
@Entity @Table



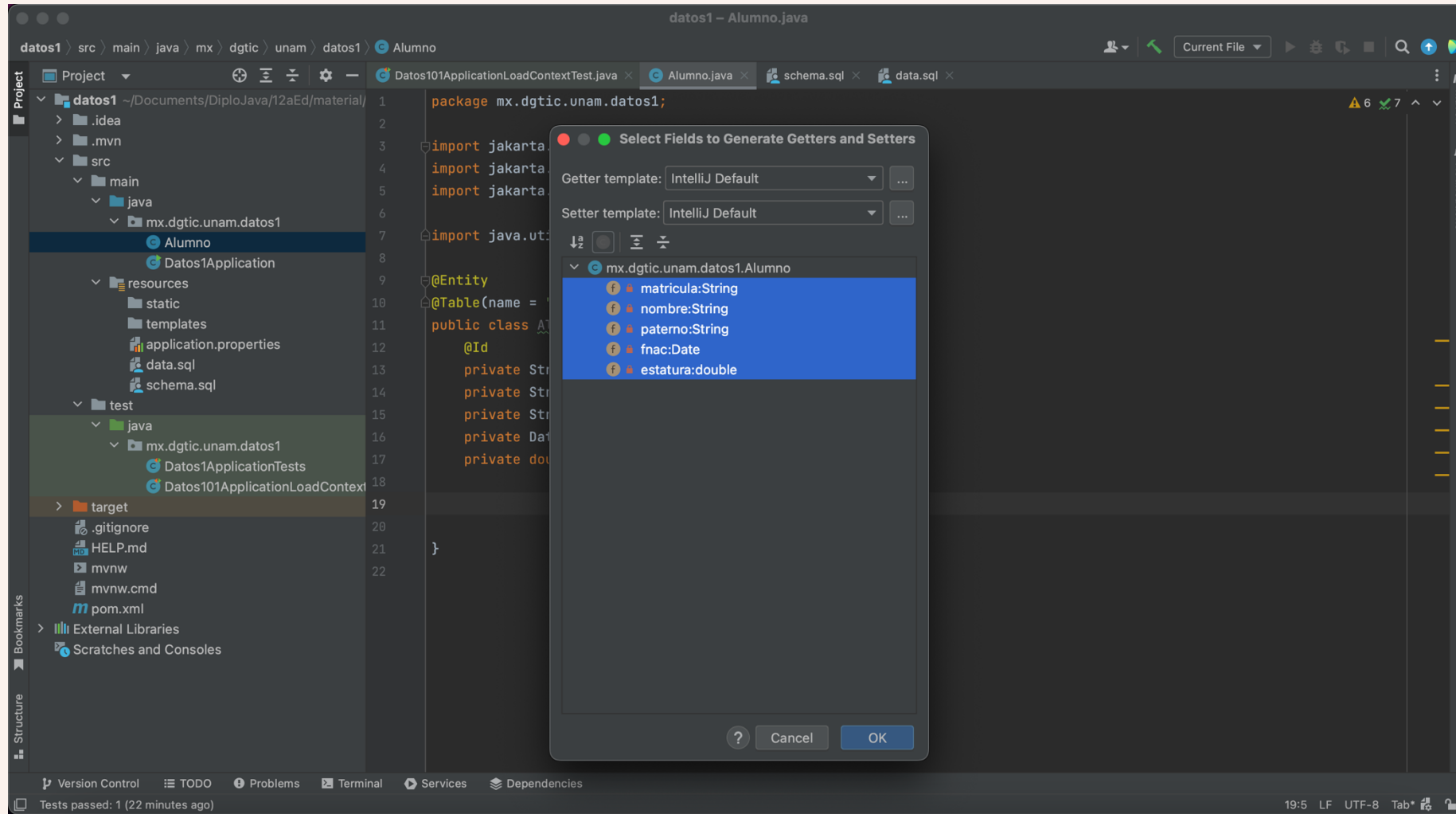
Generar Setters y Getters



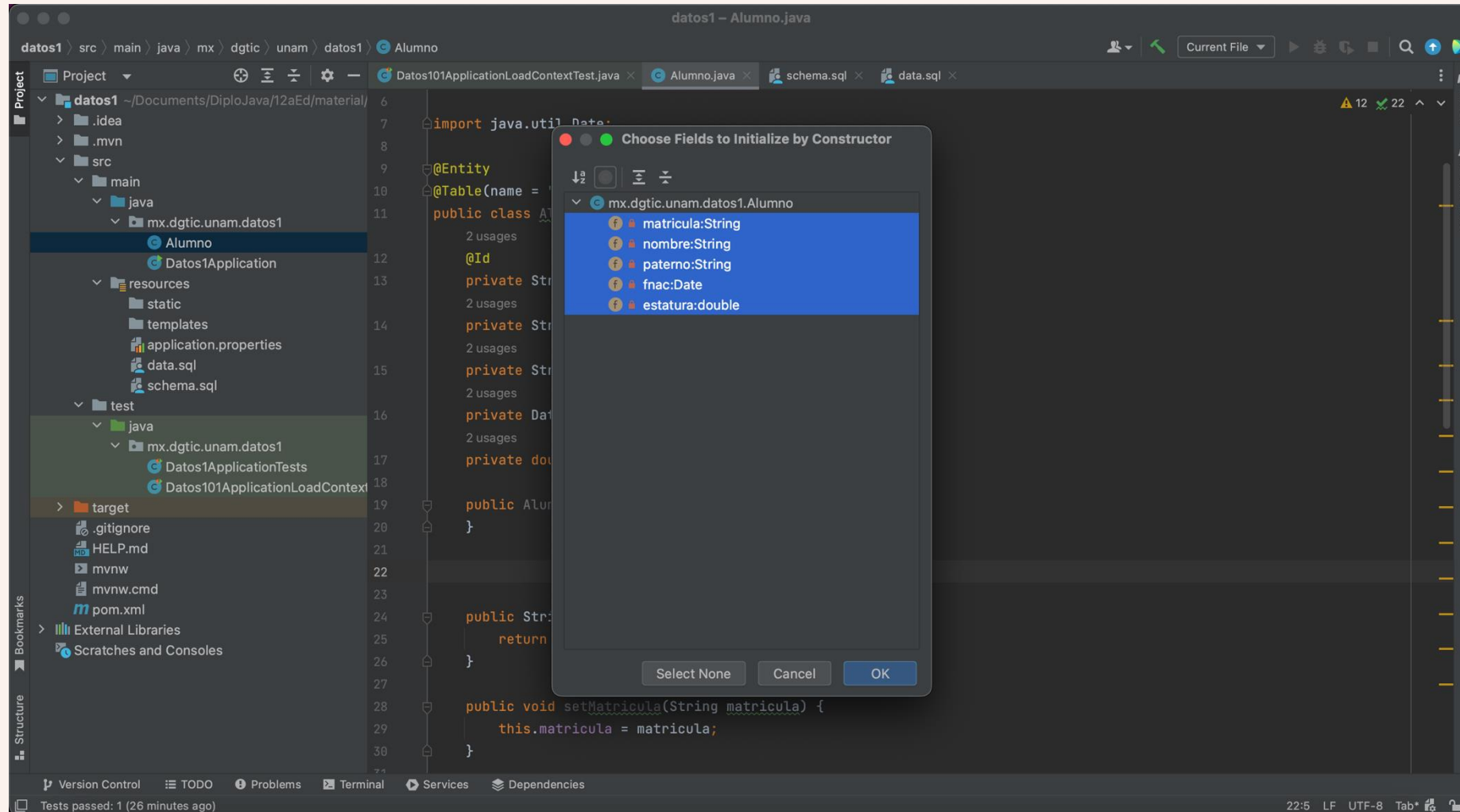
Generar Setters y Getters



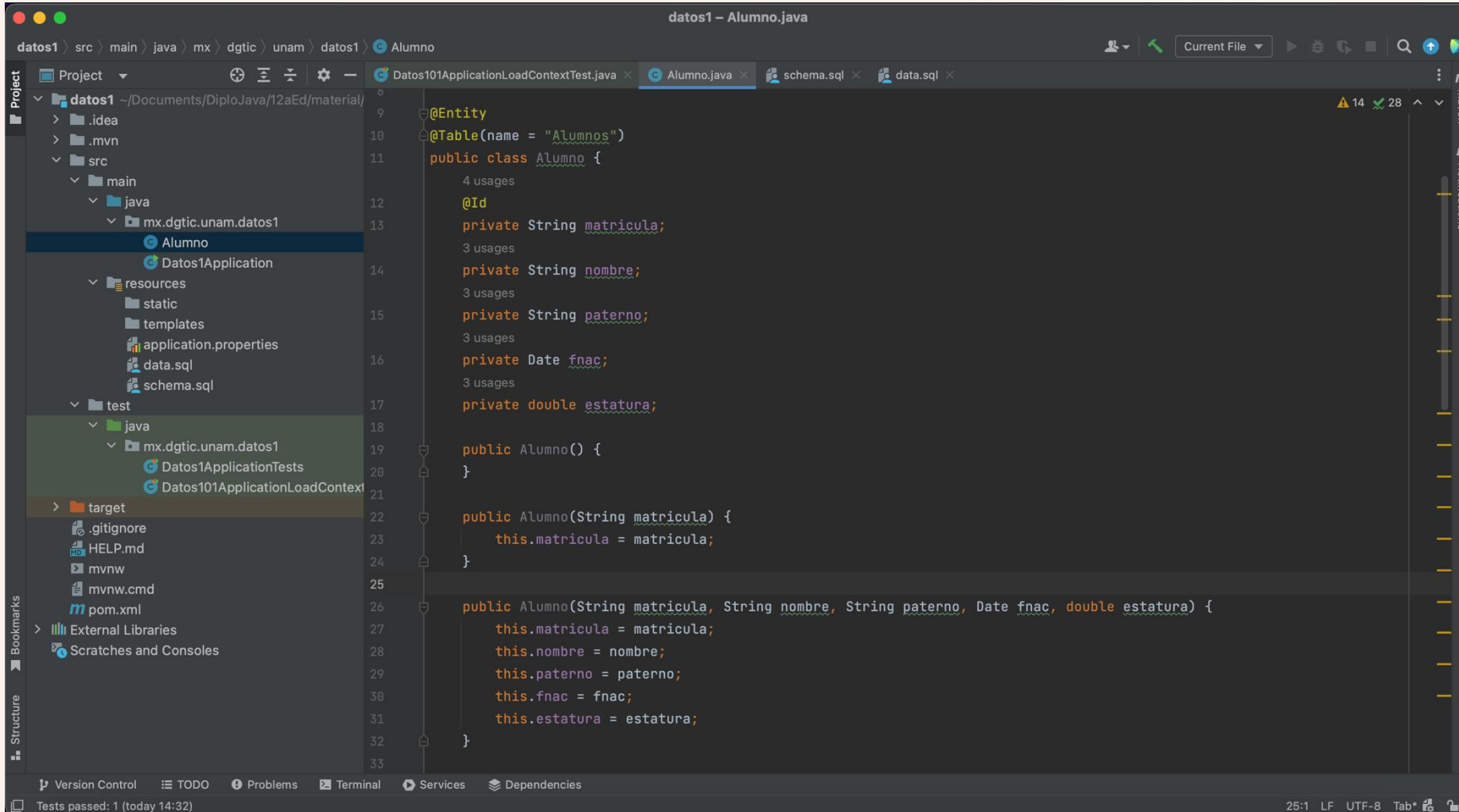
Generar Setters y Getters



Generar constructores



Generar constructores

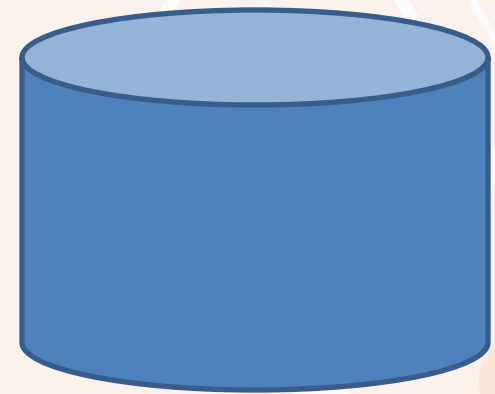


The screenshot shows an IDE window titled "datos1 - Alumno.java". The left sidebar displays a project structure for "datos1" located at "~\Documents\DiploJava\12aEd\material". The main editor area shows the code for the "Alumno" class, which is annotated with JPA annotations: `@Entity` and `@Table(name = "Alumnos")`. The class is defined as `public class Alumno` and includes several private attributes: `matricula`, `nombre`, `paterno`, `fnac`, and `estatura`. Three constructors are provided: a no-argument constructor, a constructor for `matricula`, and a full constructor for all attributes. The bottom status bar indicates "Tests passed: 1 (today 14:32)" and "25:1 LF UTF-8 Tab".

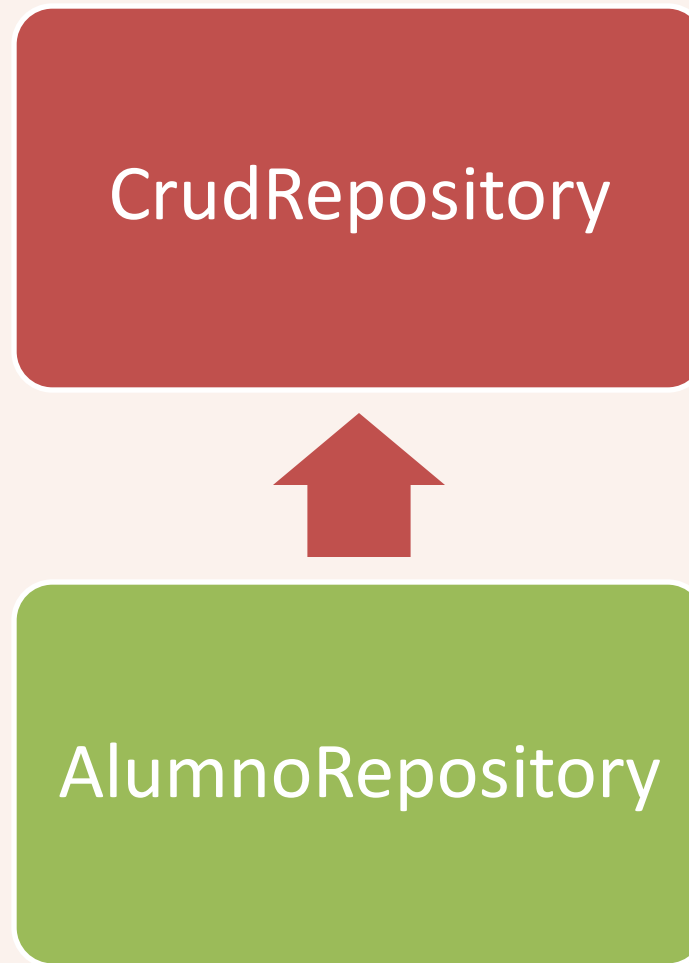
```
9  @Entity
10 @Table(name = "Alumnos")
11 public class Alumno {
12     4 usages
13     @Id
14     private String matricula;
15     3 usages
16     private String nombre;
17     3 usages
18     private String paterno;
19     3 usages
20     private Date fnac;
21     3 usages
22     private double estatura;
23
24     public Alumno() {
25     }
26
27     public Alumno(String matricula) {
28         this.matricula = matricula;
29     }
30
31     public Alumno(String matricula, String nombre, String paterno, Date fnac, double estatura) {
32         this.matricula = matricula;
33         this.nombre = nombre;
34         this.paterno = paterno;
35         this.fnac = fnac;
36         this.estatura = estatura;
37     }
38 }
```

Repositorio

AlumnoRepository



Repositorio



Spring Data

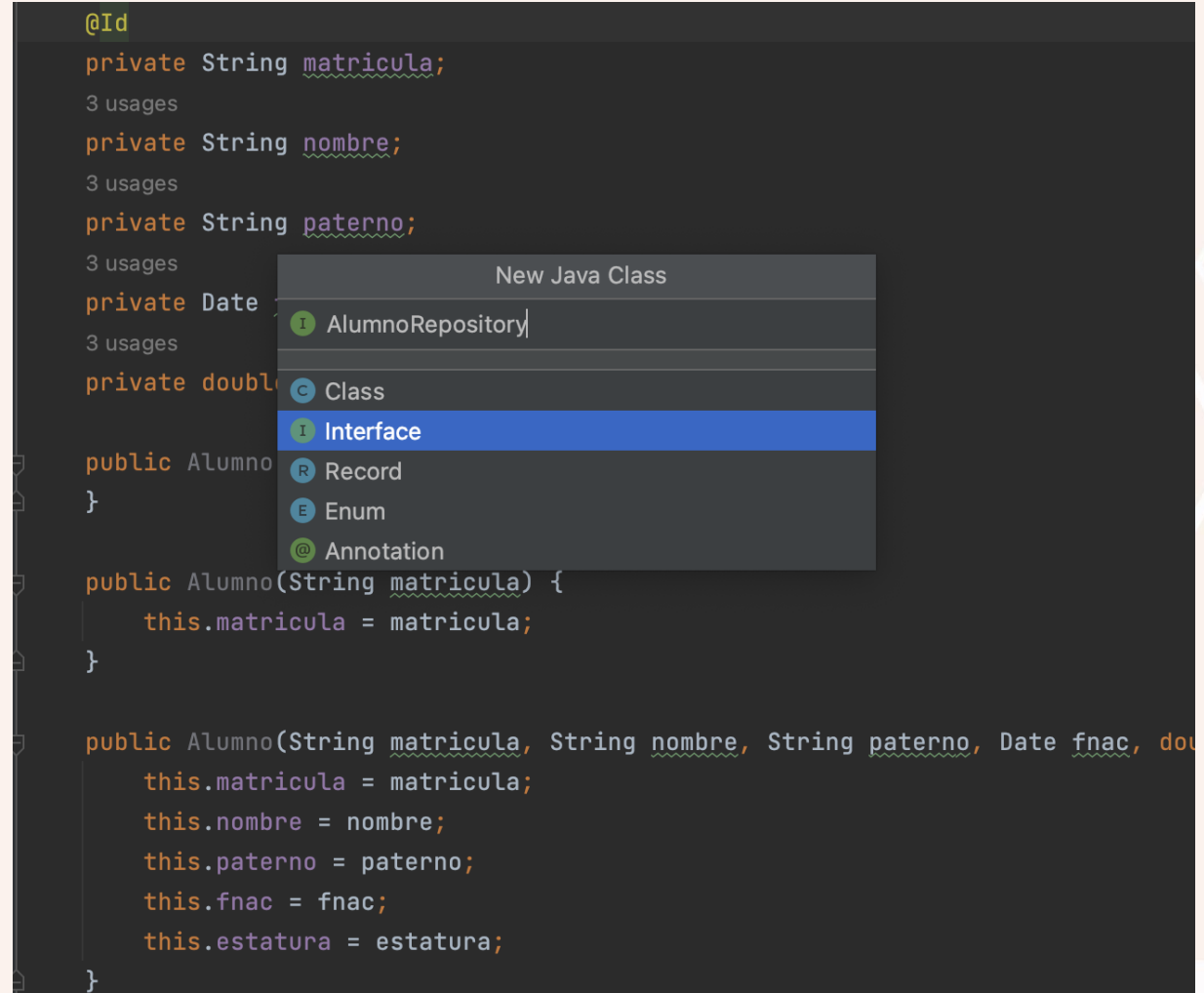
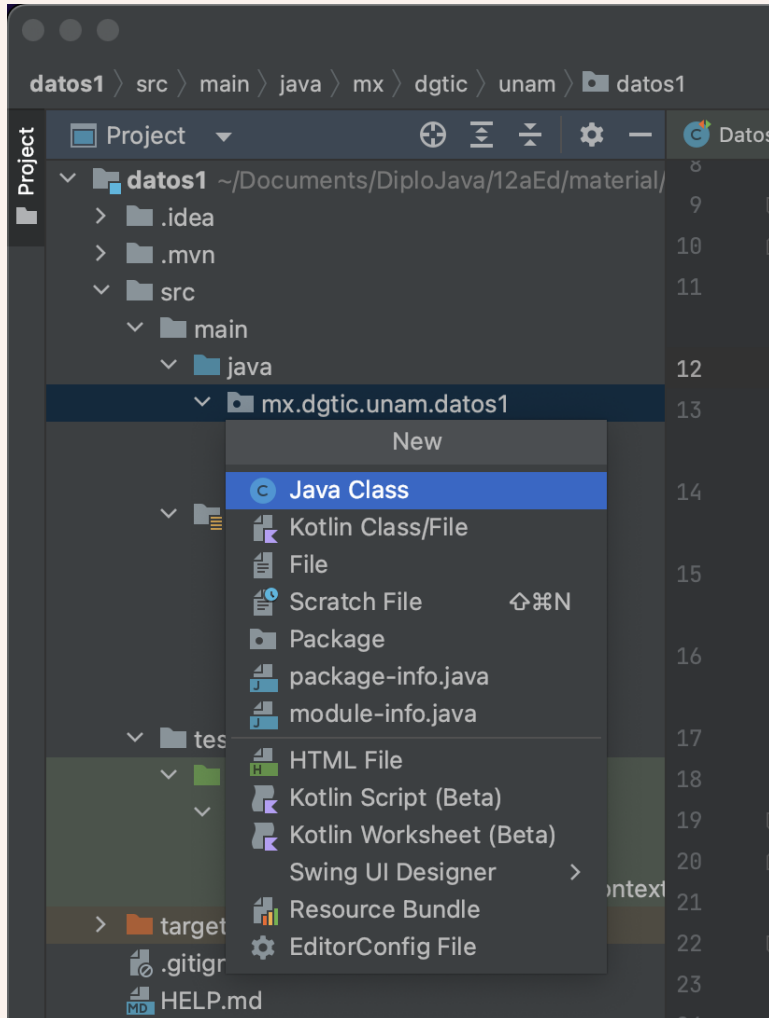
CrudRepository

```
public interface CrudRepository<T, ID> extends Repository<T, ID> {  
    <S extends T> S save(S entity);  
    Optional<T> findById(ID primaryKey);  
    Iterable<T> findAll();  
    long count();  
    void delete(T entity);  
    boolean existsById(ID primaryKey);  
    // ... more functionality omitted.  
}
```

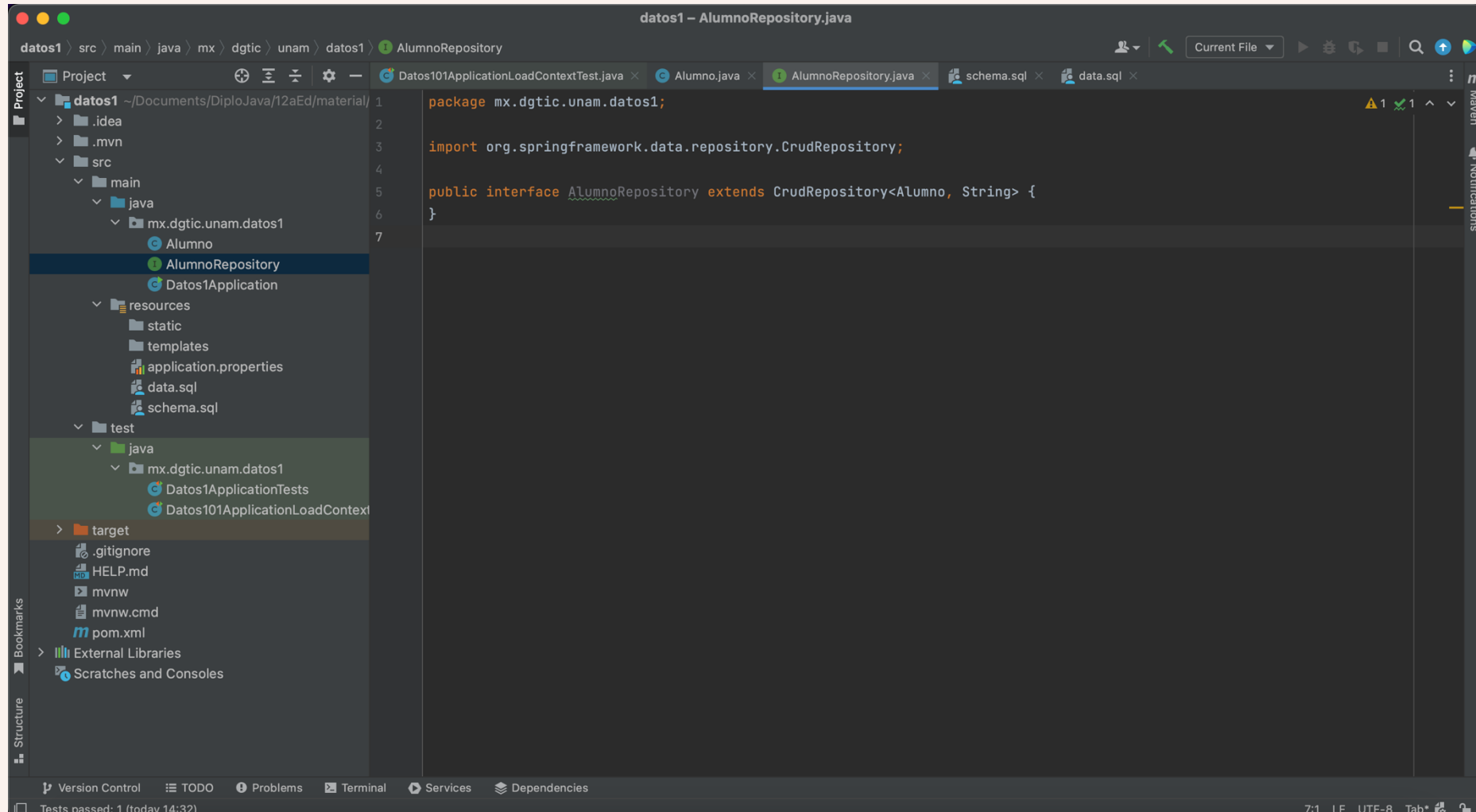
CrudRepository

Modifier and Type	Method and Description
long	<code>count()</code> Returns the number of entities available.
void	<code>delete(T entity)</code> Deletes a given entity.
void	<code>deleteAll()</code> Deletes all entities managed by the repository.
void	<code>deleteAll(Iterable<? extends T> entities)</code> Deletes the given entities.
void	<code>deleteAllById(Iterable<? extends ID> ids)</code> Deletes all instances of the type T with the given IDs.
void	<code>deleteById(ID id)</code> Deletes the entity with the given id.
boolean	<code>existsById(ID id)</code> Returns whether an entity with the given id exists.
<code>Iterable<T></code>	<code>findAll()</code> Returns all instances of the type.
<code>Iterable<T></code>	<code>findAllById(Iterable<ID> ids)</code> Returns all instances of the type T with the given IDs.
<code>Optional<T></code>	<code>findById(ID id)</code> Retrieves an entity by its id.
<code><S extends T></code> <code>S</code>	<code>save(S entity)</code> Saves a given entity.
<code><S extends T></code> <code>Iterable<S></code>	<code>saveAll(Iterable<S> entities)</code> Saves all given entities.

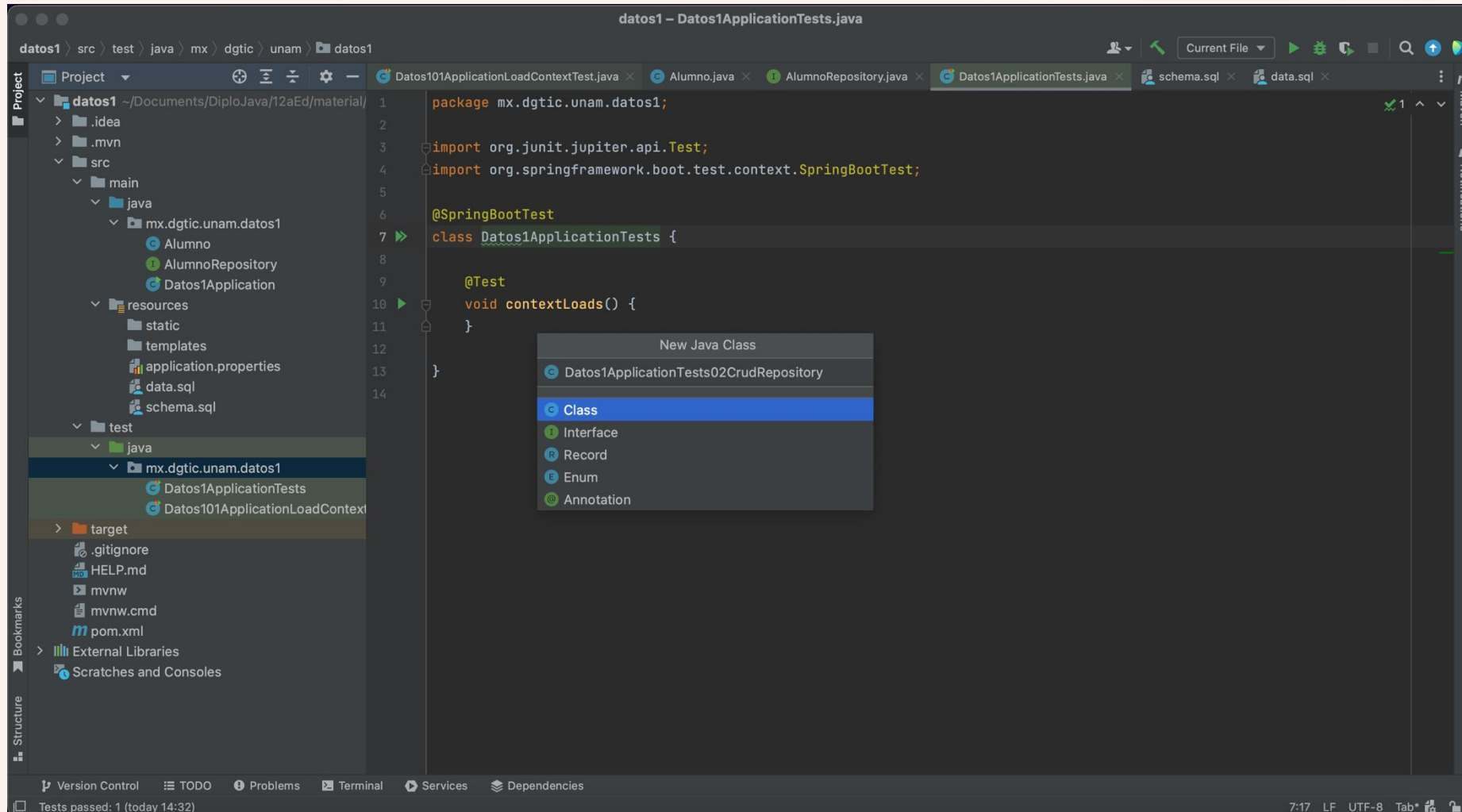
Agregar AlumnoRepository



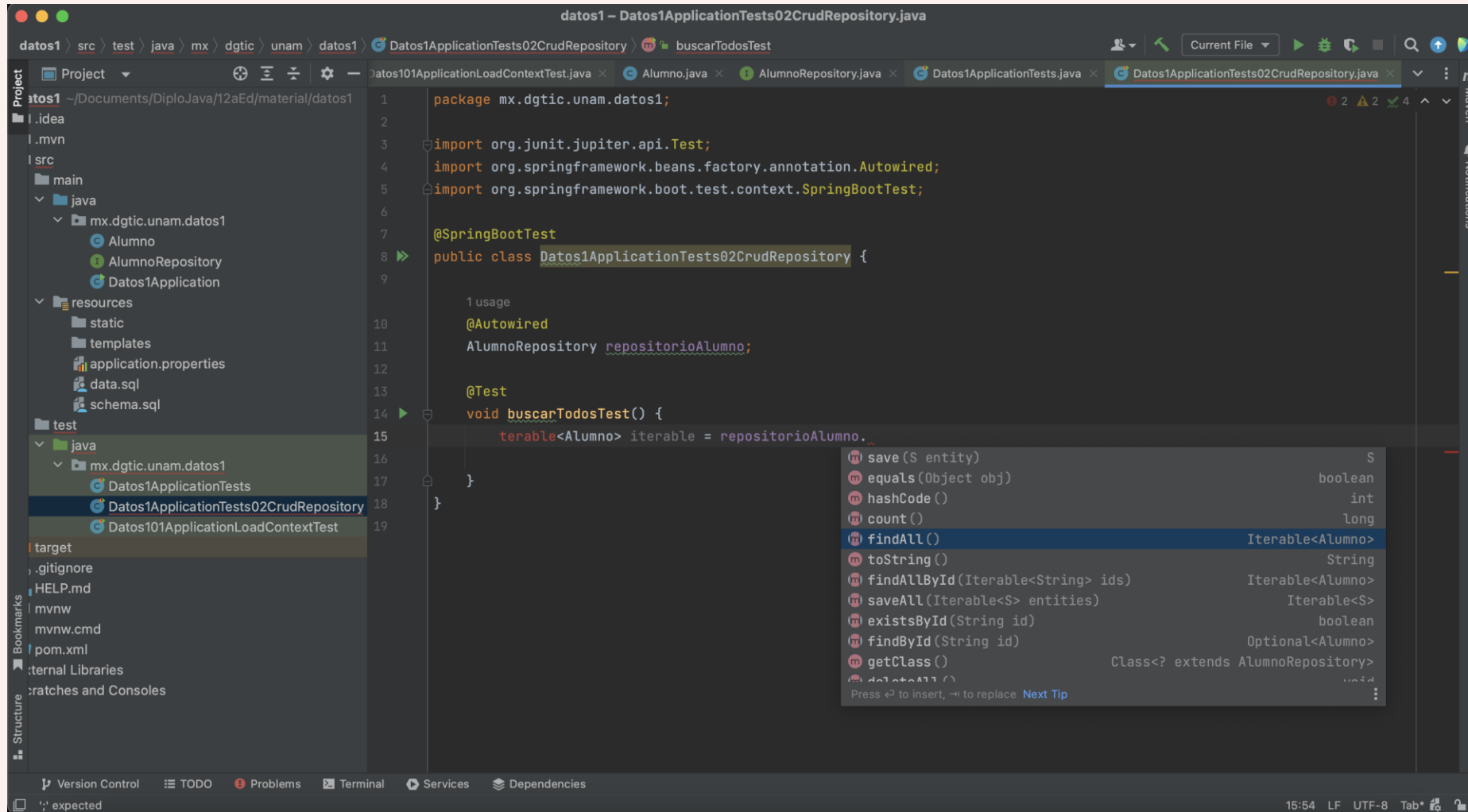
extends CrudRepository



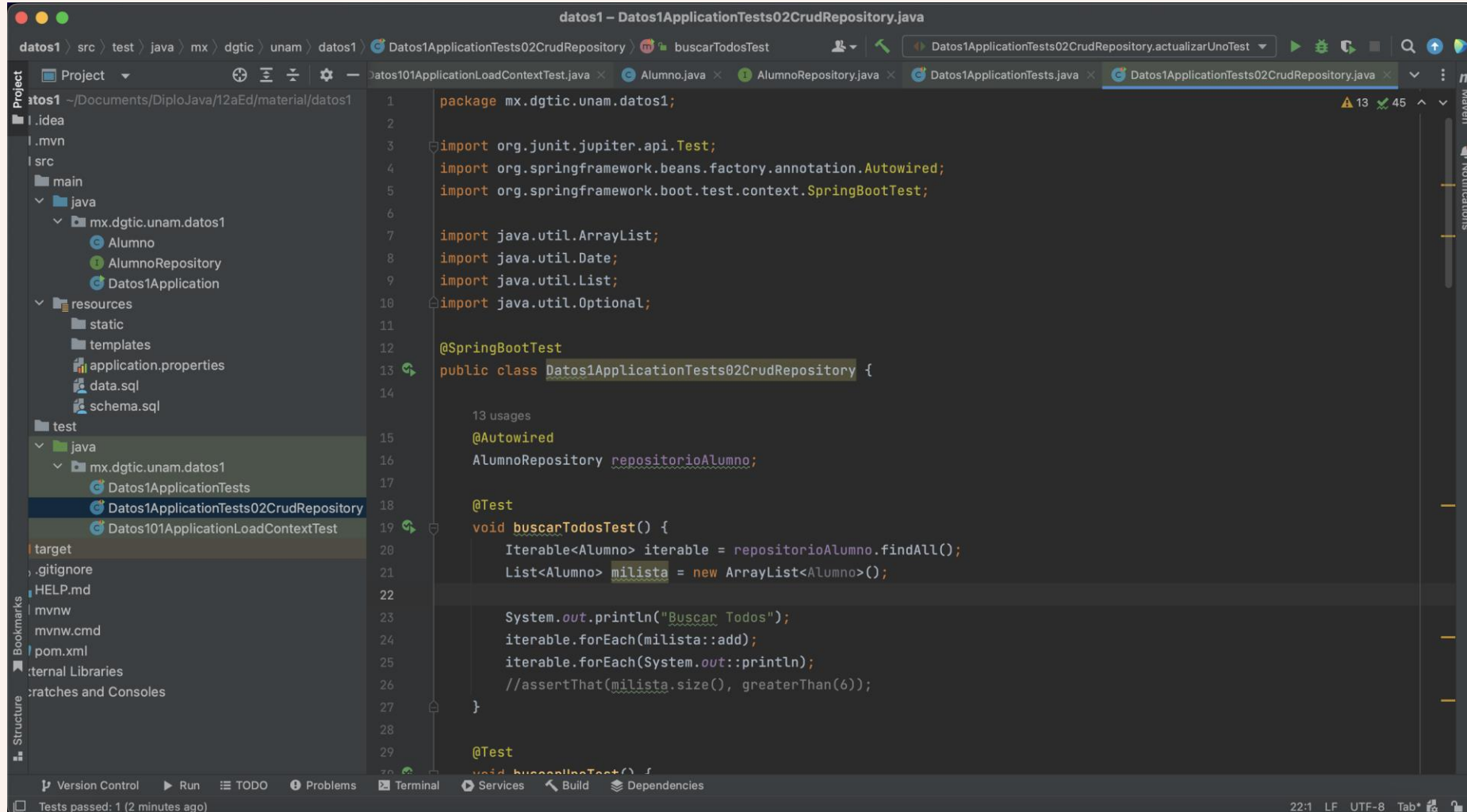
Agregar una nueva JUnit



@Autowired



Probar findAll()



The screenshot shows an IDE window titled "datos1 - Datos1ApplicationTests02CrudRepository.java". The left sidebar displays the project structure for "datos1", including "src", "resources", and "test". The "test" directory is expanded, showing "java" and "mx.dgtic.unam.datos1". The "mx.dgtic.unam.datos1" directory is selected, showing "Datos1ApplicationTests", "Datos1ApplicationTests02CrudRepository", and "Datos101ApplicationLoadContextTest". The main editor displays the code for "Datos1ApplicationTests02CrudRepository.java". The code includes imports for JUnit, Spring Framework, and Java utility classes. It defines a class "Datos1ApplicationTests02CrudRepository" with an "Autowired" "AlumnoRepository" and a "Test" method "buscarTodosTest()". The "buscarTodosTest()" method calls "repository.findAll()" and asserts that the size of the resulting list is greater than 6.

```
package mx.dgtic.unam.datos1;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;

import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.Optional;

@SpringBootTest
public class Datos1ApplicationTests02CrudRepository {

    13 usages
    @Autowired
    AlumnoRepository repositorioAlumno;

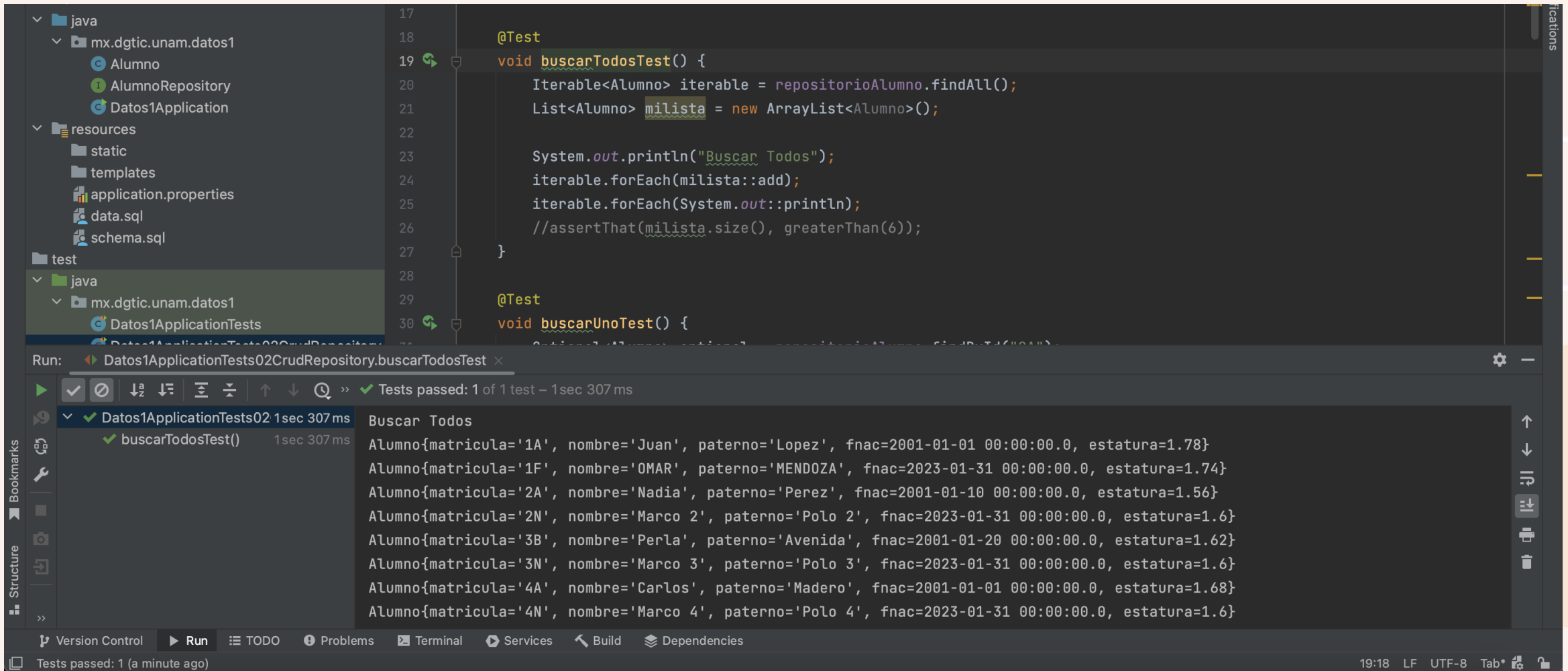
    @Test
    void buscarTodosTest() {
        Iterable<Alumno> iterable = repositorioAlumno.findAll();
        List<Alumno> milista = new ArrayList<Alumno>();

        System.out.println("Buscar Todos");
        iterable.forEach(milista::add);
        iterable.forEach(System.out::println);
        //assertThat(milista.size(), greaterThan(6));
    }

    @Test
    void buscarUnoTest() {
```

Tests passed: 1 (2 minutes ago) 22:1 LF UTF-8 Tab*

Probar findAll()



The screenshot shows an IDE with a project structure on the left, a code editor in the center, and a run console at the bottom.

Project Structure:

- java
 - mx.dgtic.unam.datos1
 - Alumno
 - AlumnoRepository
 - Datos1Application
 - resources
 - static
 - templates
 - application.properties
 - data.sql
 - schema.sql
 - test
 - java
 - mx.dgtic.unam.datos1
 - Datos1ApplicationTests

Code Editor:

```
17
18     @Test
19     void buscarTodosTest() {
20         Iterable<Alumno> iterable = repositorioAlumno.findAll();
21         List<Alumno> milista = new ArrayList<Alumno>();
22
23         System.out.println("Buscar Todos");
24         iterable.forEach(milista::add);
25         iterable.forEach(System.out::println);
26         //assertThat(milista.size(), greaterThan(6));
27     }
28
29     @Test
30     void buscarUnoTest() {
```

Run Console:

Run: Datos1ApplicationTests02CrudRepository.buscarTodosTest x

Tests passed: 1 of 1 test – 1sec 307 ms

✓ Datos1ApplicationTests02 1sec 307 ms

✓ buscarTodosTest() 1sec 307 ms

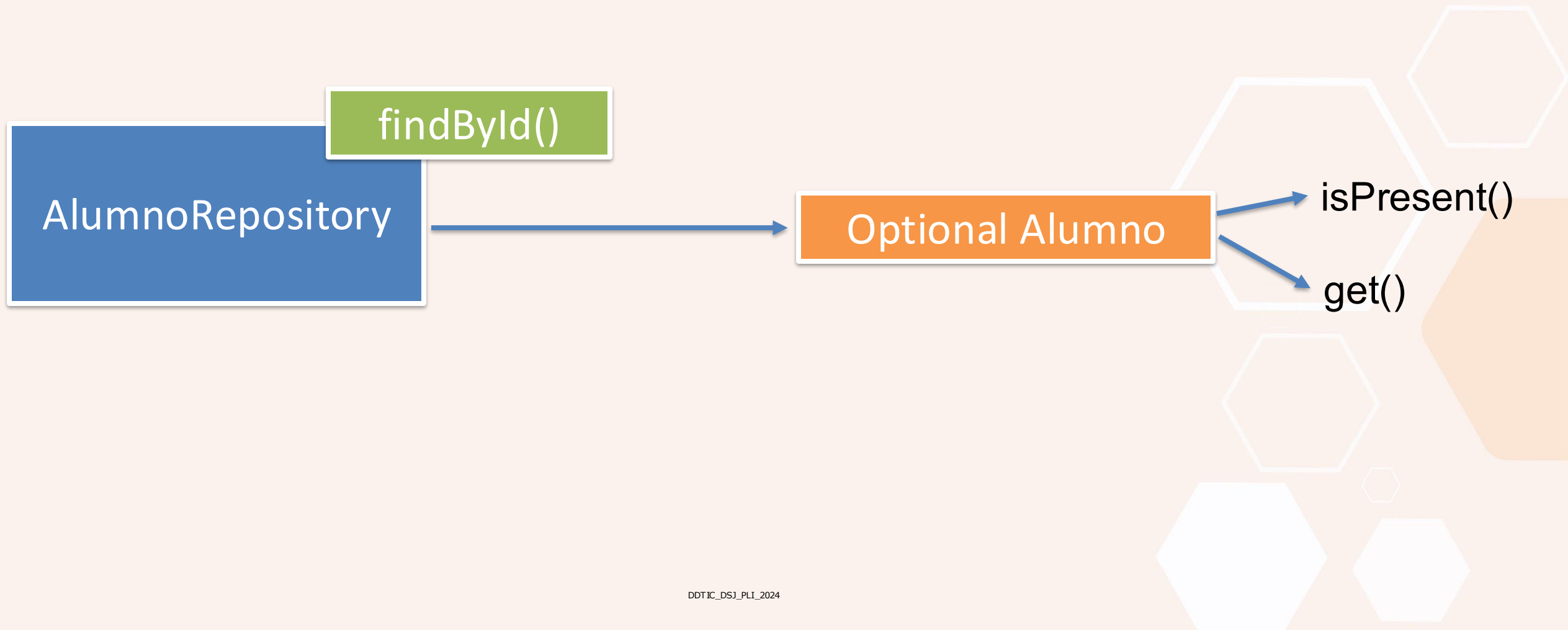
Buscar Todos

```
Alumno{matricula='1A', nombre='Juan', paterno='Lopez', fnac=2001-01-01 00:00:00.0, estatura=1.78}
Alumno{matricula='1F', nombre='OMAR', paterno='MENDOZA', fnac=2023-01-31 00:00:00.0, estatura=1.74}
Alumno{matricula='2A', nombre='Nadia', paterno='Perez', fnac=2001-01-10 00:00:00.0, estatura=1.56}
Alumno{matricula='2N', nombre='Marco 2', paterno='Polo 2', fnac=2023-01-31 00:00:00.0, estatura=1.6}
Alumno{matricula='3B', nombre='Perla', paterno='Avenida', fnac=2001-01-20 00:00:00.0, estatura=1.62}
Alumno{matricula='3N', nombre='Marco 3', paterno='Polo 3', fnac=2023-01-31 00:00:00.0, estatura=1.6}
Alumno{matricula='4A', nombre='Carlos', paterno='Madero', fnac=2001-01-01 00:00:00.0, estatura=1.68}
Alumno{matricula='4N', nombre='Marco 4', paterno='Polo 4', fnac=2023-01-31 00:00:00.0, estatura=1.6}
```

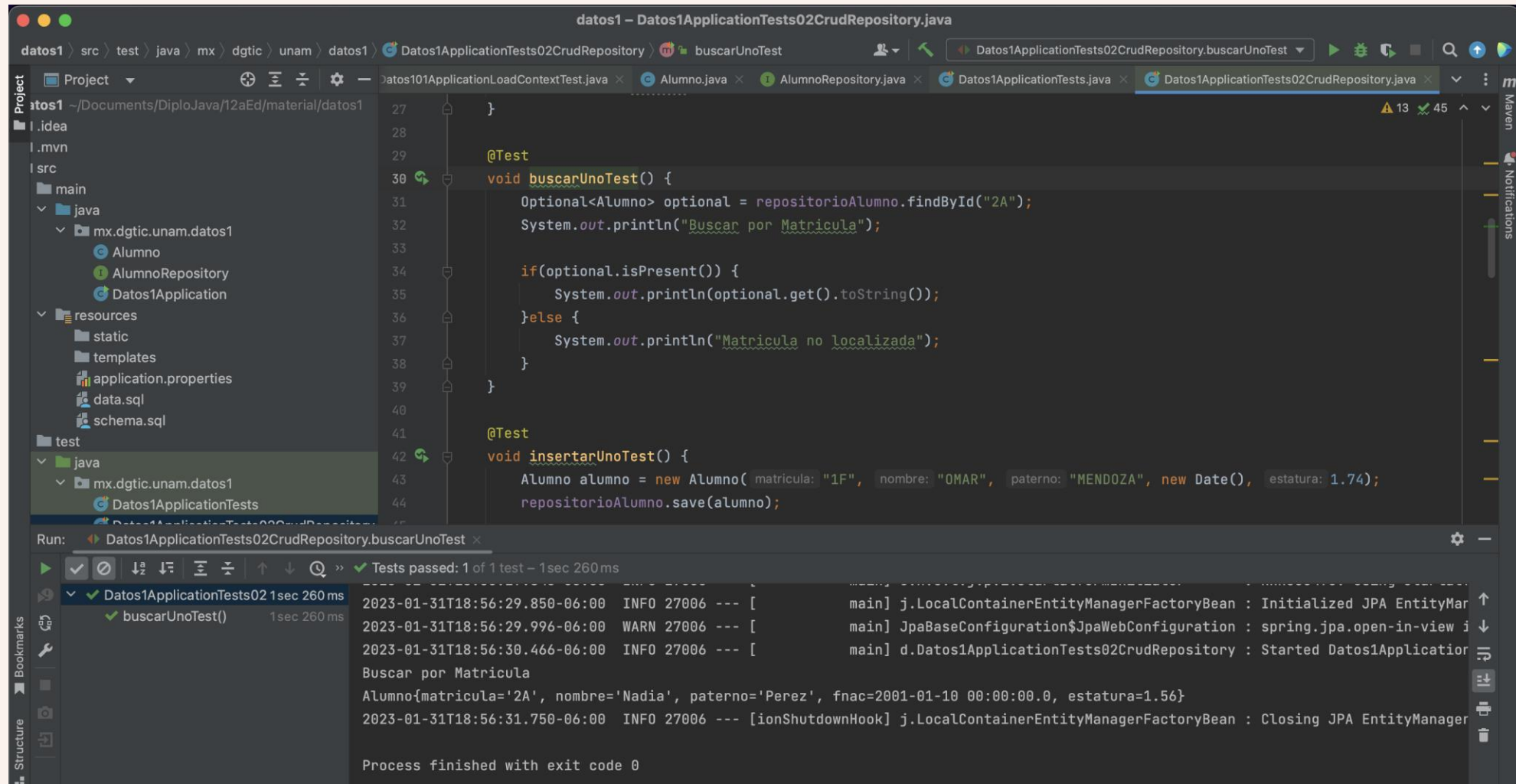
Version Control | Run | TODO | Problems | Terminal | Services | Build | Dependencies

Tests passed: 1 (a minute ago) 19:18 LF UTF-8 Tab*

Probar findById()



Probar findById()



The screenshot shows an IDE window titled "datos1 - Datos1ApplicationTests02CrudRepository.java". The editor displays the following code:

```
27     }
28
29     @Test
30     void buscarUnoTest() {
31         Optional<Alumno> optional = repositorioAlumno.findById("2A");
32         System.out.println("Buscar por Matricula");
33
34         if(optional.isPresent()) {
35             System.out.println(optional.get().toString());
36         } else {
37             System.out.println("Matricula no localizada");
38         }
39     }
40
41     @Test
42     void insertarUnoTest() {
43         Alumno alumno = new Alumno( matricula: "1F", nombre: "OMAR", paterno: "MENDOZA", new Date(), estatura: 1.74);
44         repositorioAlumno.save(alumno);
45     }
```

The "Run" tab at the bottom shows the execution of the test "Datos1ApplicationTests02CrudRepository.buscarUnoTest". The output is as follows:

```
2023-01-31T18:56:29.850-06:00 INFO 27006 --- [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory
2023-01-31T18:56:29.996-06:00 WARN 27006 --- [main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. This will result in a significant performance degradation, please use spring.jpa.open-in-view=false to disable this behavior.
2023-01-31T18:56:30.466-06:00 INFO 27006 --- [main] d.Datos1ApplicationTests02CrudRepository : Started Datos1Application
Buscar por Matricula
Alumno{matricula='2A', nombre='Nadia', paterno='Perez', fnac=2001-01-10 00:00:00.0, estatura=1.56}
2023-01-31T18:56:31.750-06:00 INFO 27006 --- [ionShutdownHook] j.LocalContainerEntityManagerFactoryBean : Closing JPA EntityManagerFactory
Process finished with exit code 0
```


Contacto

M. en C. Jesús Hernández Cabrera
Profesor de carrera

jesushc@unam.mx

Redes sociales:



www.linkedin.com/in/hcjesus